

ROBOTICS · LESSON 2

Servo With a Guard

The knob can ask for any angle — the guard decides how far the arm is actually allowed to go. The clamp you saw on an LED, now in real motion.

ROBO-002 · ~35 MIN · BEGINNER

1. The one idea

A guard sits between **what is asked** and **what is actuated**. The knob can command any angle from 0 to 180°; the guard clamps that command to the arm's safe range, so the servo only ever drives where it is allowed to go.

2. Why it matters

Every real joint has a travel limit. A gripper that closes too far crushes itself; an arm that swings past its stop strips a gear or snaps a linkage. A controller that *can* command past the limit eventually will — on a bad sensor reading, a wild input, a bug. The guard is the layer that stops "the command asked for it" from being the reason the hardware gets hurt.

You built `input → map → actuate` in Lesson 1. This adds the one piece that makes it safe to point at a real mechanism:

```
input → map → GUARD → actuate
```

3. Parts & wiring

Nothing new. This is the exact bench from Lesson 1 — the whole lesson is one idea in firmware, not new hardware.

- ESP32 dev board
- SG90 micro servo — signal on **GPIO13**, power on **external 5V**
- B10K potentiometer — wiper on **GPIO34**, outer pins to 3V3 and GND
- Shared ground between the 5V supply and the ESP32

If you finished Servo From a Pot, you are already wired for this. The shared-ground and external-5V rules from Lesson 1 §3 still hold — if the arm twitches or the board resets, fix power before you suspect the guard.

4. Predict (do this before uploading)

Pick a safe ceiling for the arm — say 120° . Now guess:

- At what knob position does the arm stop moving?
- When you push the knob *past* that point, what will the serial print show for `requested` versus `safe`?
- At the limit, does the arm drift back, hold steady, or jitter?

Write the numbers down. §6 checks them.

5. Build it — the guard is one line

Start from the Lesson 1 sketch. The only new idea is the clamp:

```

#include <ESP32Servo.h>

Servo armServo;
const int POT_PIN = 34;
const int SERVO_PIN = 13;

const int SAFE_MAX_DEG = 120;    // the guard: the arm's safe travel ceiling

void setup() {
    Serial.begin(115200);
    armServo.attach(SERVO_PIN);
}

void loop() {
    int raw = analogRead(POT_PIN);
    int requested = map(raw, 0, 4095, 0, 180);    // what the knob asks for
    int safe = min(requested, SAFE_MAX_DEG);    // the guard: clamp to the safe
    limit

    bool clamped = requested > SAFE_MAX_DEG;

    armServo.write(safe);    // drive the SAFE angle, never the
    raw demand

    Serial.printf("req=%3d  safe=%3d  %s\n", requested, safe, clamped ? "CLAMP" :
    "pass");
    delay(15);
}

```

The whole guard is `int safe = min(requested, SAFE_MAX_DEG);` — plus the discipline of writing `safe`, not `requested`, to the servo. That second part is the lesson: a guard you compute but don't actuate is decoration. The guard only protects the mechanism if the mechanism is driven by the guarded value.

This is the same contract as `threshold_reflex_v0` — `safe = min(requested, limit)` — the rule the LED Reflex Guard and every ARTY thermometer course run. You met it clamping an LED's brightness; here it clamps a motor's travel.

6. Test against your prediction

Open the Serial Monitor at 115200 baud and turn the knob slowly. Below 120° the two numbers track and the arm follows your hand. Past 120°, `requested` keeps climbing toward 180 but `safe` pins at 120 and the line flips to `CLAMP` — and the arm just **stops**, holding at the limit while your fingers keep turning.

That gap between `req` and `safe`, printed live, is the guard doing its job. The clamp you could only see on an LED bar, you now *feel* in the mechanism: the knob says more, the arm says no.

7. What goes wrong (pre-loaded debugging)

- **Arm still drives into the stop / buzzes at the end** → you wrote `requested` to the servo instead of `safe`. Computing the clamp isn't enough — you have to actuate the clamped value. This is the single most common guard bug.
- **Arm stops too early or too late** → `SAFE_MAX_DEG` doesn't match the joint's real travel. Measure where the mechanism actually binds and set the limit below it, with margin.
- **You never see `CLAMP`** → the pot isn't sweeping its full range, or the limit is ≥ 180 so the demand never exceeds it. Lower `SAFE_MAX_DEG` and try again.
- **Servo strains hard at the limit** → expected if it's holding right at the clamp, but if it's grinding, your safe limit is past the real mechanical stop — back it off.
- **Twitch or reset** → the Lesson 1 power gotchas (brownout, common ground), not the guard.

*Honest boundary: this guard clamps the **command**. It does not stop an external force from back-driving the joint — push the arm by hand and it still moves. The guard governs what you tell the servo, not what the world does to it. A real safety system adds sensing and interlocks on top; here we guard the command path, honestly and only that.*

8. Checkpoint

You now have a guard between demand and actuation: the knob can ask for anything, the arm only ever goes where it is allowed. That is the reflex-guard contract — the same one running on the LED and on the FPGA — now clamping real motion. Every robot that doesn't tear itself apart has this layer somewhere; you just built the smallest honest version of it.

9. Push further

- **A floor as well as a ceiling:** `safe = constrain(requested, SAFE_MIN_DEG, SAFE_MAX_DEG)` for a two-sided safe band.

- **A slew-rate limit:** cap how many degrees the command may move per loop, so the arm can't snap from 0 to the limit in one step. That is the `maxStep` field of the same `threshold_reflex_v0` kernel — the next piece of the contract.
- **Latch the violation:** light an LED the first time a clamp happens and keep it lit until a button clears it — exactly like the ARTY violation latch (LD1). Now a breach stays on the record.
- **Lost-signal safe pose:** if the pot reading goes stale or out of range, drive the arm to a known-safe angle instead of the last command.

These are Lesson 3 territory — and they are the same fields and behaviors the LED and FPGA guards already use, now on a moving joint.